

NavLink v2 — Protocol Specification

Wire format, dialect, codegen contract, services, security & time sync

Vayu flight stack · `src/comm`, `software/src/protocol`, `tools/navlink`

June 2026

Abstract

The normative specification for **NavLink v2**, the Vayu flight stack’s link protocol. It defines the byte-exact frame format, the CRC-16/**CRC_EXTRA** integrity scheme, field encoding and packed-struct layout, the JSON message dialect and its schema, the code-generation contract (wire/aligned structs, converters, dispatcher), the command/parameter/mission/file services, the AEAD security layer, the wall-clock time-synchronisation protocol, the full message catalog with its core/vendor msgid split, the v1 v2 migration, and conformance test vectors. The companion document *NavLink v2 — Protocol Analysis & Redesign* (`navlink-v2-design.md`) gives the rationale; this document gives the rules.

Contents

1. Scope, conformance, and conventions	2
1.1 Scope	2
1.2 Requirement levels	3
1.3 Conventions	3
1.4 Versioning	3
2. Terminology	3
3. Frame format	3
3.1 Overview	3
3.2 Header layout (normative)	4
3.3 incompat_flags (normative)	4
3.4 Version demultiplexing (v1 / v2 coexistence)	4
3.5 Sequence number	5
3.6 Sizes	5
4. Integrity: CRC-16 and CRC_EXTRA	5
4.1 CRC-16/MCRF4XX parameters	5
4.2 Frame checksum computation (unsecured frames)	5
4.3 CRC_EXTRA (per-message layout signature)	6
5. Field encoding	6
5.1 Scalar types	6
5.2 Wire order — declaration order, no reordering	6
5.3 Packed layout & padding	7
5.4 Fixed-point fields	7
5.5 Extension fields (forward/backward growth)	7
5.6 Trailing-zero truncation	7
6. Time and timestamp model	7
6.1 Wall-clock field [s:20 ms:12]	7
6.2 High-rate sample time	8
7. The dialect (single source of truth)	8
7.1 Authoring rules (normative)	8
7.2 Message object schema	8

7.3 Worked example dialect	9
7.4 Dialect JSON Schema (validation in CI)	11
8. Code-generation contract	12
8.1 What is generated	12
8.2 The two representations	13
8.3 The dispatcher	13
9. Message-ID space	14
10. Time-synchronisation protocol	15
10.1 Principle	15
10.2 Messages	15
10.3 Algorithm	15
10.4 Schedule	15
10.5 Security gate (normative)	15
11. Security layer	15
11.1 Modes	15
11.2 Secured trailer	16
11.3 Cipher	16
11.4 Nonce construction	16
11.5 Replay protection (reboot-safe)	16
11.6 Order of operations	16
11.7 Keys	16
11.8 Downgrade protection	17
12. Services	17
12.1 Command service	17
12.2 Parameter service	17
12.3 Mission service	17
12.4 Bulk transfer	17
13. Capability negotiation	17
14. Message catalog	18
14.1 Core / system (0–255)	18
14.2 Sensors / state (1024–2047)	18
14.3 Navigation / position (2048–3071)	19
14.4 Power / propulsion (3072–4095)	19
14.5 Peripherals / payload (4096–8191)	19
14.6 Commands (8192–12287)	19
14.7 Services (12288–16383)	19
15. Migration (v1 v2)	20
16. Conformance & test vectors	20
16.1 CRC-16/MCRF4XX self-check	20
16.2 CRC_EXTRA worked example	20
16.3 Unsecured frame round-trip	20
16.4 Secured frame	21
16.5 Version demux	21
17. Appendix — reference enumerations	21

1. Scope, conformance, and conventions

1.1 Scope

This document specifies NavLink v2 completely enough to write an interoperable implementation in firmware (C, STM32F4), the ground station (C++), and tools (Python). It covers the wire format, the integrity and security layers, the dialect that is the single source of truth, the code generated from it, the service protocols, and the migration from v1.

1.2 Requirement levels

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHOULD**, **SHOULD NOT**, **MAY**, and **OPTIONAL** are to be interpreted as in RFC 2119.

1.3 Conventions

- All multi-byte integers are **little-endian** on the wire unless stated.
- Bit 0 is the least-significant bit.
- Floating point is **IEEE-754** (f32 = binary32, f64 = binary64).
- Hex literals are written 0x...; byte sequences are space-separated hex.
- A B denotes concatenation; X[a..b] is bytes a inclusive to b exclusive; X[a..b] with both inclusive is written X[a ... b] in prose.
- “FC” = flight controller; “GCS” = ground control station; “peer” = either.

1.4 Versioning

This document specifies wire **version 0x02** (the value carried in header byte 1, §3.2). The protocol identity is otherwise NavLink’s own; NavLink v2 is *not* wire-compatible with MAVLink and interoperability with any external protocol is a non-goal.

2. Terminology

Term	Meaning
frame	one on-wire unit: header + payload + trailer (§3)
msgid	24-bit message type identifier (§9)
dialect	the JSON catalog of all messages; single source of truth (§7)
CRC_EXTRA	1-byte per-message layout signature folded into the checksum (§4.3)
wire struct	packed C/C++ struct matching the payload byte-for-byte (§8.2)
aligned struct	naturally-aligned application struct for the same message (§8.2)
secured frame	a frame carrying IFLAG_SIGNED or IFLAG_ENCRYPTED (§11)
T0	the synchronised reference epoch for wall-clock timestamps (§10)
skew	the tolerated FC GCS clock disagreement for secured frames (§11.5)

3. Frame format

3.1 Overview

A NavLink v2 frame is:

+	-----+	-----+	-----+	+
sync	header (bytes 1 ... 9)	payload	trailer	
0x56	version,len,flags,seq,...	0 ... 255 bytes	CRC or secured	
+	-----+	-----+	-----+	+

The header is a **fixed 10 bytes** (offsets 0–9). The trailer is either a 2-byte CRC (unsecured) or a [timestamp:6] [tag:N] secured trailer (§11).

3.2 Header layout (normative)

off	size	field	value / meaning
0	1	sync	0x56 ('V'). Frame start. MUST be 0x56.
1	1	version	0x02 for this spec. Demuxes v1/v2 (§3.4).
2	1	payload_len	number of payload bytes on the wire, 0..255 (§5.6 truncation)
3	1	incompat_flags	unknown bit set receiver MUST drop the frame (§3.3)
4	1	seq	per-(sysid,compid) frame counter, rolls 0..255 (§3.5)
5	1	sysid	sender system id, 1..255 (0 = anonymous/broadcast)
6	1	compid	sender component id within the system
7	3	msgid	message type, u24 little-endian, 0..16,777,215 (§9)
10	N	payload	payload_len bytes, packed wire layout (§5)
10+N	2	checksum	CRC-16/MCRF4XX seeded with CRC_EXTRA (§4) - UNSECURED only

There is **no compat_flags** byte and **no timestamp** in the header; the former is replaced by capability negotiation (§13), the latter lives only in the payloads that need it (§10).

3.3 incompat_flags (normative)

incompat_flags advertises frame-structural features that a receiver **MUST** understand to parse the frame at all. **If a receiver sees any bit set that it does not implement, it MUST silently drop the frame** (it cannot be parsed safely). Framing is not lost: **payload_len** lets the parser skip the frame and resynchronise.

Bit	Name	Effect
0x01	IFLAG_SIGNED	secured trailer present, payload authenticated (cleartext) — §11
0x02	IFLAG_ENCRYPTED	secured trailer present, payload encrypted+authenticated — §11
0x04	IFLAG_CRC32	bulk mode: 4-byte CRC-32 trailer instead of CRC-16 — §12.4
0x08	IFLAG_FRAGMENTED	payload is a fragment of a larger message — §12.4.1
0x10–0x80	reserved	MUST be 0 in this version

IFLAG_SIGNED and IFLAG_ENCRYPTED are mutually exclusive on a single frame (encryption implies authentication). When either is set, the 2-byte CRC is **absent** and replaced by the secured trailer (§11.2).

There is exactly **one** incompat byte (8 bits). It is the one header field whose exhaustion would force a frame-format bump, so it is deliberately a full byte; advisory, layout-neutral capabilities are negotiated once at connect (§13) rather than carried per frame.

3.4 Version demultiplexing (v1 / v2 coexistence)

v1 and v2 share the 0x56 sync byte. A receiver distinguishes them on **byte 1**:

```

read byte0
if byte0 != 0x56: resync (scan forward to next 0x56)
read byte1
if (byte1 & 0x0F) == 0x01:  parse as NavLink v1    # v1 packs (type<<4)|version, version==1
elif byte1 == 0x02:        parse as NavLink v2
else:                      resync

```

This is unambiguous: every v1 frame has low-nibble 1 in byte 1 (its version field), and 0x02 never occurs there for v1. A future v3 takes 0x03, etc.

3.5 Sequence number

`seq` increments by 1 (mod 256) for every frame a peer emits **per** (`sysid`, `compid`). Receivers MAY use gaps to estimate link loss. `seq` participates in the AEAD nonce for secured frames (§11.4); a sender MUST NOT emit two secured frames with the same (`sysid`, `compid`, `seq`, `timestamp`).

3.6 Sizes

Quantity	Value
sync	1 byte, 0x56
header	10 bytes
payload	0–255 bytes (single frame); larger via §12.4
unsecured trailer	2 bytes (CRC-16) or 4 bytes (IFLAG_CRC32)
secured trailer	6 + N bytes, N ∈ {8, 16} (§11.2)
min frame	12 bytes (10 header + 0 payload + 2 CRC)
max unsecured frame	267 bytes (10 + 255 + 2)

4. Integrity: CRC-16 and CRC_EXTRA

4.1 CRC-16/MCRF4XX parameters

Parameter	Value
width	16
polynomial	0x1021 (reflected 0x8408)
init	0xFFFF
reflect in/out	true / true
xorout	0x0000
check ("123456789")	0x6F91

Reference accumulator (one byte at a time; identical to MAVLink’s `crc_accumulate`, reproduced normatively):

```
static inline void crc_accumulate(uint8_t b, uint16_t *crc) {
    uint8_t t = b ^ (uint8_t)(*crc & 0xFF);
    t ^= (t << 4);
    *crc = (*crc >> 8) ^ ((uint16_t)t << 8) ^ ((uint16_t)t << 3) ^ ((uint16_t)t >> 4);
}
```

4.2 Frame checksum computation (unsecured frames)

```
crc = 0xFFFF
for b in header[1 .. 9]:          # version, payload_len, incompat_flags, seq,
    crc_accumulate(b, &crc)        # sysid, compid, msgid[0..3] - everything after sync
for b in payload[0 .. payload_len]:
    crc_accumulate(b, &crc)
crc_accumulate(CRC_EXTRA[msgid], &crc)  # the per-message seed (§4.3)
checksum = crc                      # little-endian into trailer bytes
```

The `sync` byte (offset 0) is **excluded**. The `CRC_EXTRA` seed is accumulated **last**. On `IFLAG_CRC32` frames the same input is run through CRC-32 (§12.4). Secured frames carry no CRC; integrity is the AEAD tag (§11).

A receiver **MUST** recompute and compare; a mismatch **MUST** cause the frame to be dropped. Unlike v1, a checksum of 0x0000 is **valid** and **MUST NOT** be special- cased.

4.3 CRC_EXTRA (per-message layout signature)

CRC_EXTRA[msgid] is a 1-byte value computed by codegen from the message's *structure*, so that a sender and receiver that disagree on a message's layout produce different seeds and the checksum **fails** rather than silently mis-decoding.

Algorithm (over **declaration/index order**, which equals wire order, §5.2; extension fields §5.5 are **excluded**):

```
crc = 0xFFFF
accumulate each ASCII byte of message_name
accumulate one space (0x20)
for field in non_extension_fields, in ascending index order:
    accumulate each ASCII byte of field.wire_type_name  # "u8","i32","f32",...
    accumulate one space (0x20)
    accumulate each ASCII byte of field.name
    accumulate one space (0x20)
    if field.is_array:
        accumulate (uint8_t) field.array_len
CRC_EXTRA = (uint8_t)((crc & 0xFF) ^ (crc >> 8))
```

wire_type_name uses the canonical names of §5.1 (a fixed-point field uses its **integer** wire type name, e.g. i32, not deg). The receiver keeps a generated msgid → CRC_EXTRA table; an unknown msgid has an unknown seed and the frame **MUST** be dropped as undecodable (framing preserved via payload_len).

5. Field encoding

5.1 Scalar types

name	wire bytes	C type	notes
u8 / i8	1	uint8_t / int8_t	
u16 / i16	2	uint16_t / int16_t	LE
u24	3	—	LE, used for msgid references
u32 / i32	4	uint32_t / int32_t	LE
u64 / i64	8	uint64_t / int64_t	LE
f32	4	float	IEEE-754 binary32, LE
f64	8	double	IEEE-754 binary64, LE
char	1	char	ASCII/UTF-8 code unit

Arrays are type[len] with a fixed len; char[len] is a fixed-width, NUL-padded (not necessarily NUL-terminated) string.

5.2 Wire order — declaration order, no reordering

Fields are emitted on the wire in **ascending index order** exactly as declared in the dialect (§7). NavLink does **not** reorder fields by size (a deliberate divergence from MAVLink). Combined with packed structs (§8.2) this makes wire order, source order, and struct layout identical.

5.3 Packed layout & padding

The generated wire struct is **packed** (no implicit padding). Padding is only present where the dialect author declares an explicit `pad` field (`{"name": "_pad", "type": "u8"}` or `"u8" [n]`). A conformant payload therefore has no compiler-dependent gaps.

5.4 Fixed-point fields

A field MAY declare an integer wire `type` plus a `scale` and `unit`, meaning the real value is `wire_value / scale` in `unit`. Use fixed-point where `f32`'s ~7-significant-digit *relative* precision is insufficient:

quantity	type	scale	unit	resolution / range
latitude/longitude	i32	1e7	deg	~1.1 cm, ±214.7°
altitude	i32	1e3	m	1 mm, ±2,147 km
horizontal/vertical speed	i16	1e2	m/s	1 cm/s, ±327 m/s
heading	u16	1e2	deg	0.01°, 0–655.35

`scale` is chosen per field so `range × scale` fits the integer width. A blanket `×1e7` is **non-conformant** (it overflows large-range fields and cannot encode `f64`). Small bounded quantities (angles in rad, rates, temperatures, voltages) stay `f32/f64`.

5.5 Extension fields (forward/backward growth)

A field MAY be flagged `extension: true`. Extension fields:

1. are serialised **after** all non-extension fields, in ascending index order;
2. are **excluded** from `CRC_EXTRA` (§4.3), so adding them does not change a message's seed and old/new peers still validate each other's frames;
3. MUST only be appended (never inserted among, or removed from, the original fields).

5.6 Trailing-zero truncation

A sender SHOULD drop trailing all-zero **plaintext** bytes and report the shortened length in `payload_len`. A receiver MUST zero-fill the payload back to the message's full known length before decoding. Consequences:

- new sender → old receiver: trailing extension bytes are simply beyond what the old receiver reads;
- old sender → new receiver: absent (new) fields read as 0, their defined “absent” value.

On secured frames, truncation runs on the plaintext **before** encryption (§11.6); `payload_len` is authenticated as AAD so the fill length cannot be tampered.

6. Time and timestamp model

6.1 Wall-clock field [s:20 | ms:12]

The standard wall-clock timestamp is a 32-bit little-endian word:

bits 31..12 : seconds (20 bits) – integer offset from synced epoch T0 (§10)
bits 11..0 : ms (12 bits) – sub-second milliseconds, 0..999

- 20-bit seconds range 2^{20} s **291 h**, far exceeding the 5 h resync interval (§10.4), so it never rolls within a session.
- 12-bit ms holds 0–999; the 2 spare high bits of the ms sub-field are reserved. Bit 11 is the **TIME_STALE** flag: the sender MUST set it before its first successful sync and after any clock discontinuity, and a receiver MUST treat a stamp with **TIME_STALE** set as unsynchronised.

6.2 High-rate sample time

High-rate sensor messages additionally carry `sample_time_us` (u32, μ s since boot, monotonic, single clock domain) for intra-vehicle alignment finer than 1 ms. This is independent of the wall-clock field and is **not** GCS-referenced.

7. The dialect (single source of truth)

The dialect is one JSON document, `navlink/dialect.json`, from which all codecs, the `CRC_EXTRA` table, and the dispatcher are generated (§8).

7.1 Authoring rules (normative)

1. **Fields are an ordered array** of objects, never a `name→type` map (JSON object key order is unspecified; wire order is load-bearing).
2. **Every field carries an explicit integer index.** Indices **MUST** be unique within a message and contiguous from 0 across the non-extension fields; extension fields continue the sequence. Codegen **MUST** reject violations.
3. **Every field declares its type** from the §5.1 enum; missing/unknown type is an error.
4. Decimal quantities that need fixed-point declare `scale` (number) and `unit` (string) alongside an integer type (§5.4).
5. Annotations go in a `doc` string per message/field (JSON has no comments); codegen emits them as `docstrings` / `// comments`.
6. 64-bit constants (e.g. defaults) are written as **hex strings** (`"0x..."`), because JSON numbers are IEEE-754 doubles (exact only to 2^3).
7. `msgid` **MUST** lie in the core half (`0x000000–0x7FFFFFFF`) for in-tree messages; vendor dialects use `0x800000–0xFFFFFFFF` (§9).

The generator auto-assigns and then **freezes** index values for newly added fields (writing them back into the file), so authors need not hand-number while the explicit index still guarantees a stable ABI.

7.2 Message object schema

```
message := {
  "msgid":    integer (0 .. 16777215),
  "name":     string  (UPPER_SNAKE, unique),
  "replaces": string  (optional, v1 cross-reference),
  "doc":      string  (optional),
  "fields":   [ field, ... ]
}
field := {
  "index":    integer (>= 0, unique within message),
  "name":     string  (lower_snake, unique within message),
  "type":     enum    ("u8"|"i8"|"u16"|"i16"|"u24"|"u32"|"i32"|"u64"|"i64"|"f32"|"f64"|"char"),
  "len":      integer (optional, >=1; makes the field an array),
  "scale":    number  (optional; fixed-point divisor),
  "unit":     string  (optional; physical unit),
  "enum":     string  (optional; references an enums[] entry),
  "default":  number|string (optional; string = hex for 64-bit),
  "extension": boolean (optional, default false),
  "doc":      string  (optional)
}
```

7.3 Worked example dialect

A self-contained fragment showing a telemetry message, a fixed-point position message with extensions, a typed command, and a typed parameter value:

```
{
  "dialect": "vayu-core",
  "version": 2,
  "enums": {
    "command_result": {
      "doc": "COMMAND_ACK.result",
      "entries": [
        { "name": "ACCEPTED", "value": 0 },
        { "name": "TEMPORARILY_REJECTED", "value": 1 },
        { "name": "DENIED", "value": 2 },
        { "name": "UNSUPPORTED", "value": 3 },
        { "name": "FAILED", "value": 4 },
        { "name": "IN_PROGRESS", "value": 5 }
      ]
    },
    "param_type": {
      "entries": [
        { "name": "U8", "value": 1 }, { "name": "I8", "value": 2 },
        { "name": "U16", "value": 3 }, { "name": "I16", "value": 4 },
        { "name": "U32", "value": 5 }, { "name": "I32", "value": 6 },
        { "name": "U64", "value": 7 }, { "name": "I64", "value": 8 },
        { "name": "F32", "value": 9 }, { "name": "F64", "value": 10 }
      ]
    }
  },
  "messages": [
    {
      "msgid": 0, "name": "HEARTBEAT", "replaces": "0x0",
      "doc": "~1 Hz liveness, mode, advertised capabilities, wall-clock stamp.",
      "fields": [
        { "index": 0, "name": "type", "type": "u8", "doc": "vehicle type" },
        { "index": 1, "name": "autopilot", "type": "u8" },
        { "index": 2, "name": "base_mode", "type": "u8" },
        { "index": 3, "name": "system_status", "type": "u8" },
        { "index": 4, "name": "nav_state", "type": "u8" },
        { "index": 5, "name": "capabilities", "type": "u32", "doc": "capability bitmask (see §13)" },
        { "index": 6, "name": "timestamp", "type": "u32", "doc": "wall-clock [s:20|ms:12] (§6.1)" }
      ]
    },
    {
      "msgid": 1026, "name": "ATTITUDE_EULER", "replaces": "0x4",
      "doc": "Euler attitude + body rates, NED frame.",
      "fields": [
        { "index": 0, "name": "roll", "type": "f32", "unit": "rad", "doc": "NED roll" },
        { "index": 1, "name": "pitch", "type": "f32", "unit": "rad" },
        { "index": 2, "name": "yaw", "type": "f32", "unit": "rad" },
        { "index": 3, "name": "rollspeed", "type": "f32", "unit": "rad/s" },
        { "index": 4, "name": "pitchspeed", "type": "f32", "unit": "rad/s" }
      ]
    }
  ]
}
```

```

    { "index": 5, "name": "yawspeed",    "type": "f32", "unit": "rad/s" }
  ]
},

{
  "msgid": 2048, "name": "GPS_RAW_INT", "replaces": null,
  "doc": "Raw GNSS fix. Position is fixed-point (§5.4) - f32 loses >1 m globally.",
  "fields": [
    { "index": 0, "name": "fix_type",    "type": "u8" },
    { "index": 1, "name": "satellites",  "type": "u8" },
    { "index": 2, "name": "lat",          "type": "i32", "scale": 1e7, "unit": "deg" },
    { "index": 3, "name": "lon",          "type": "i32", "scale": 1e7, "unit": "deg" },
    { "index": 4, "name": "alt",          "type": "i32", "scale": 1e3, "unit": "m", "doc": "AMSL" },
    { "index": 5, "name": "eph",          "type": "u16", "scale": 1e2, "unit": "m" },
    { "index": 6, "name": "epv",          "type": "u16", "scale": 1e2, "unit": "m" },
    { "index": 7, "name": "vel",          "type": "u16", "scale": 1e2, "unit": "m/s" },
    { "index": 8, "name": "cog",          "type": "u16", "scale": 1e2, "unit": "deg" },
    { "index": 9, "name": "sample_time_us", "type": "u32", "unit": "us" },

    { "index": 10, "name": "alt_ellipsoid", "type": "i32", "scale": 1e3, "unit": "m",
      "extension": true, "doc": "added later; old peers ignore it (§5.5)" }
  ]
},

{
  "msgid": 8195, "name": "CMD_SET_PID",
  "doc": "Typed command (§12.1). Opens with the standard command header.",
  "fields": [
    { "index": 0, "name": "target_sys",  "type": "u8" },
    { "index": 1, "name": "target_comp",  "type": "u8" },
    { "index": 2, "name": "req_seq",      "type": "u8", "doc": "echoed in COMMAND_ACK" },
    { "index": 3, "name": "axis",         "type": "u8", "doc": "0=roll 1=pitch 2=yaw" },
    { "index": 4, "name": "kp",           "type": "f32" },
    { "index": 5, "name": "ki",           "type": "f32" },
    { "index": 6, "name": "kd",           "type": "f32" }
  ]
},

{
  "msgid": 5, "name": "COMMAND_ACK", "replaces": "0x3",
  "doc": "Acknowledges any command, correlated by (command, req_seq).",
  "fields": [
    { "index": 0, "name": "command",      "type": "u24", "doc": "acked command's msgid" },
    { "index": 1, "name": "req_seq",      "type": "u8" },
    { "index": 2, "name": "result",       "type": "u8",  "enum": "command_result" },
    { "index": 3, "name": "progress",     "type": "u8",  "doc": "0..100 for IN_PROGRESS" },
    { "index": 4, "name": "result_param2", "type": "i32" }
  ]
},

{
  "msgid": 12290, "name": "PARAM_VALUE",
  "doc": "Typed parameter value (§12.2). 'value' is the real type, never a float bit-cast.",
  "fields": [

```

```

    { "index": 0, "name": "param_id",    "type": "char", "len": 16 },
    { "index": 1, "name": "index",      "type": "u16" },
    { "index": 2, "name": "count",      "type": "u16" },
    { "index": 3, "name": "generation", "type": "u32", "doc": "bumped on every PARAM_SET (§12.2)" },
    { "index": 4, "name": "type",       "type": "u8",  "enum": "param_type" },
    { "index": 5, "name": "value",      "type": "u8",  "len": 8, "doc": "real value, LE, in 'type'",
  }
]
}
}

```

7.4 Dialect JSON Schema (validation in CI)

The dialect MUST validate against this schema (Draft 2020-12) before codegen; CI additionally checks index contiguity, msgid uniqueness, msgid range, and `scale-fits-width`:

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://vayu/navlink/dialect.schema.json",
  "type": "object",
  "required": ["dialect", "version", "messages"],
  "properties": {
    "dialect": { "type": "string" },
    "version": { "const": 2 },
    "enums": {
      "type": "object",
      "additionalProperties": {
        "type": "object",
        "required": ["entries"],
        "properties": {
          "doc": { "type": "string" },
          "entries": {
            "type": "array",
            "items": {
              "type": "object",
              "required": ["name", "value"],
              "properties": {
                "name": { "type": "string", "pattern": "^[A-Z][A-Z0-9_]*$" },
                "value": { "type": "integer", "minimum": 0 },
                "doc": { "type": "string" }
              }
            },
            "additionalProperties": false
          }
        }
      },
      "additionalProperties": false
    },
    "messages": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["msgid", "name", "fields"],
        "properties": {
          "msgid": { "type": "integer", "minimum": 0, "maximum": 16777215 },

```

```

"name":      { "type": "string", "pattern": "^[A-Z][A-Z0-9_]*$" },
"replaces":  { "type": ["string", "null"] },
"doc":       { "type": "string" },
"fields": {
  "type": "array",
  "minItems": 1,
  "items": {
    "type": "object",
    "required": ["index", "name", "type"],
    "properties": {
      "index": { "type": "integer", "minimum": 0 },
      "name": { "type": "string", "pattern": "^[a-z_][a-z0-9_]*$" },
      "type": { "enum": ["u8", "i8", "u16", "i16", "u24", "u32", "i32",
        "u64", "i64", "f32", "f64", "char"] },
      "len": { "type": "integer", "minimum": 1 },
      "scale": { "type": "number", "exclusiveMinimum": 0 },
      "unit": { "type": "string" },
      "enum": { "type": "string" },
      "default": { "type": ["number", "string"] },
      "extension": { "type": "boolean" },
      "doc": { "type": "string" }
    },
    "additionalProperties": false
  },
  "additionalProperties": false
},
"additionalProperties": false
}

```

8. Code-generation contract

tools/navlink/generate.py reads dialect.json and emits, deterministically:

- firmware: include/comm/navlink_msgs.h, src/comm/navlink_msgs.c
- GCS: software/src/protocol/NavlinkMsgs.{h,cpp}
- tools: tools/autotune/navlink_msgs.py

8.1 What is generated

1. **Enums** as C enum / C++ enum class / Python IntEnum.
2. **Two structs per message** (§8.2).
3. **pack / unpack** between a byte buffer and the wire struct.
4. **to_aligned / from_aligned** converters (§8.2).
5. **The CRC_EXTRA table** (msgid → u8) and a msgid → name/size table.
6. **The dispatcher** (§8.3).

The generated v1 codec MUST be byte-identical to the legacy hand codec (golden-vector test against captured v_nav.bin).

8.2 The two representations

For each message M, codegen emits a **packed wire struct** matching the wire byte-for-byte and a **natural-aligned application struct**:

```
/* wire image - exact bytes, no padding */
typedef struct __attribute__((packed)) {
    uint8_t  axis;
    float    kp;           /* offset 1 - unaligned, that is fine for a packed struct */
    float    ki;
    float    kd;
} navlink_cmd_set_pid_wire_t;   /* 13 bytes */

/* application struct - naturally aligned, padded by the compiler */
typedef struct {
    uint8_t  axis;
    float    kp, ki, kd;
} navlink_cmd_set_pid_t;
```

Codec (wire bytes): one memcpy. Because the wire struct is packed and both ends are little-endian (and floats are IEEE-754 both sides), the whole payload moves in a single copy:

```
size_t navlink_cmd_set_pid_pack(uint8_t *buf, const navlink_cmd_set_pid_wire_t *w) {
    memcpy(buf, w, sizeof *w);           /* + trailing-zero truncation, §5.6 */
    return sizeof *w;
}

void navlink_cmd_set_pid_unpack(navlink_cmd_set_pid_wire_t *w,
                               const uint8_t *buf, size_t len) {
    memset(w, 0, sizeof *w);             /* zero-fill for truncation */
    memcpy(w, buf, len <= sizeof *w ? len : sizeof *w);
}
```

The codec **MUST NOT** form a typed pointer into the packed struct (&w->kp); taking the address of a packed member is undefined behaviour on strict targets. Whole-struct memcpy and member reads (which the compiler emits unaligned-safe code for) are the only permitted accesses.

Converters (wire aligned): field-wise, lazy. Application code that does arithmetic uses the aligned struct, reached via generated converters:

```
void navlink_cmd_set_pid_to_aligned(navlink_cmd_set_pid_t *a,
                                    const navlink_cmd_set_pid_wire_t *w) {
    a->axis = w->axis; a->kp = w->kp; a->ki = w->ki; a->kd = w->kd;
}
```

Conversion is OPTIONAL per use site: pure forward/log/relay paths stay on the wire struct and skip it. The converters are also the single place a byte-swap would live if a big-endian host ever appeared (none today).

8.3 The dispatcher

Codegen emits a msgid → { decode, handler-slot } table so dispatch is table-driven, not hand-written branching. On the FC:

```
typedef void (*navlink_handler_fn)(const navlink_frame_t *hdr, const void *aligned_msg);

typedef struct {
    uint32_t      msgid;
    uint16_t      wire_size;
    uint8_t       crc_extra;
```

```

    navlink_handler_fn handler;      /* registered by the application */
} navlink_dispatch_entry_t;

extern navlink_dispatch_entry_t navlink_dispatch_table[]; /* generated, sorted by msgid */

/* generated dispatch: O(log n) binary search on msgid */
int navlink_dispatch(const navlink_frame_t *hdr, const uint8_t *payload, size_t len);

```

navlink_dispatch looks up `hdr->msgid`, validates CRC_EXTRA, unpacks into the wire struct, `to_aligneds`, and calls the registered handler (`on_CMD_SET_PID(const navlink_cmd_set_pid_t*)`). This replaces the v1 hand-written if/else on packet type, the nested if/else on `cmd_id`, and the GCS `std::variant` switch. The generator SHOULD warn for any msgid that has no registered handler so a new message cannot be silently dropped.

9. Message-ID space

The 24-bit space (16,777,216 ids) is split on its **top bit**:

Range	Owner	Rule
0x000000–0x7FFFFFFF	core / FC	assigned only in-tree (this spec)
0x800000–0xFFFFFFFF	vendor / experimental	(msgid & 0x800000) != 0; core never assigns here

A vendor therefore picks ids freely in the upper half and can never collide with a future official message. The core half is laid out in spaced blocks:

Range (dec)	hex	Block
0–255	0x000000–0x0000FF	Core / system
256–1023	0x000100–0x0003FF	reserved (core growth)
1024–2047	0x000400–0x0007FF	Sensors / state
2048–3071	0x000800–0x000BFF	Navigation / position
3072–4095	0x000C00–0x000FFF	Power / propulsion
4096–8191	0x001000–0x001FFF	Peripherals / payload
8192–12287	0x002000–0x002FFF	Commands (typed)
12288–16383	0x003000–0x003FFF	Services (param/mission/file/cal)
16384–65535	0x004000–0x00FFFF	reserved (near-term core)
65536–8388607	0x010000–0x7FFFFFFF	deep future core reserve (~8.3 M)

The full catalog with fields is §14.

10. Time-synchronisation protocol

10.1 Principle

There is no clock in the header. The FC never invents wall-clock time; the GCS establishes the reference epoch **T0** and the FC reports **now - T0** in the [s:20|ms:12] field (§6.1). The sync is a GCS-initiated **round trip** that compensates for link latency (one-way delay at 115200 baud 1.4 ms the 1 ms LSB, so it cannot be ignored).

10.2 Messages

```
TIME_REFERENCE      { epoch_unix_s: u32, gcs_send_us: u64 }      # GCS → FC
TIME_REFERENCE_ACK  { gcs_send_us: u64, fc_recv_us: u64, fc_send_us: u64 } # FC → GCS
```

10.3 Algorithm

Four timestamps are recorded: **t1** = GCS send (**gcs_send_us**), **t2** = FC receive (**fc_recv_us**), **t3** = FC send of ack (**fc_send_us**), **t4** = GCS receive of ack. The GCS computes (NTP-style):

```
rtt    = (t4 - t1) - (t3 - t2)
delay  = rtt / 2
offset = ((t2 - t1) + (t3 - t4)) / 2
```

The GCS then sends a second **TIME_REFERENCE** whose **epoch_unix_s** is corrected by **delay**; the FC latches its local tick counter to it and thereafter emits **now - T0**.

Note (asymmetry). **delay** = **rtt**/2 assumes symmetric up/down latency — true for a USB/UART bench link, approximate on an asymmetric telemetry radio. The **offset** term and the periodic/emergency resyncs (§10.4) bound the residual.

10.4 Schedule

1. **Startup** — at link establishment, before any FC stamp is trusted.
2. **Periodic** — every **5 h** of continuous operation (bounds crystal drift; a few-ppm TCXO drifts a handful of ms over 5 h, inside the 1 ms LSB).
3. **Emergency** — the GCS compares incoming FC stamps to its own clock and pushes an out-of-schedule **TIME_REFERENCE** if the error exceeds threshold.

Because each resync re-establishes **T0** near zero, the 20-bit seconds field never approaches its 291 h ceiling.

10.5 Security gate (normative)

Until the startup sync succeeds — and whenever the FC's clock and the GCS reference disagree by more than **skew** (§11.5) — the FC is **unsynchronised**: it **MUST** set **TIME_STALE** (§6.1) on its own stamps and **MUST** reject every secured and every command frame (§11, §12.1). This is what allows the synchronised wall-clock to serve as the anti-replay anchor (§11.5) with no persisted counter.

11. Security layer

11.1 Modes

Per-frame, selected by **incompat_flags** (§3.3), default-off, gated at build time by **ENABLE_NAVLINK_CRYPT**O and advertised in **CAPABILITIES** (§13):

- **IFLAG_SIGNED** — authenticate only; payload cleartext.
- **IFLAG_ENCRYPTED** — AEAD; payload encrypted **and** authenticated.

Both use one **AEAD** construction (no separate encrypt-then-MAC). The header is the AEAD **associated data** (authenticated, not encrypted) so addressing stays readable for routing/framing; only the payload is encrypted (for `IFLAG_ENCRYPTED`).

11.2 Secured trailer

When a security flag is set the 2-byte CRC is **absent**, replaced by:

[timestamp : 6] [tag : N] N = 16 (full Poly1305/GCM) or 8 (truncated)

- **timestamp** — 48-bit, 100 μ s ticks since synced epoch T0 (§10), strictly monotonic per (sysid, compid, direction).
- **tag** — AEAD tag over (AAD ciphertext), where AAD = header bytes 1 ... 9.

11.3 Cipher

Default **ChaCha20-Poly1305** (IETF, 96-bit nonce, 128-bit tag): fast and constant-time in software on the Cortex-M4, no crypto peripheral required (F405/ F407 have none; only F415/417/437/439 ship CRYIP). **AES-GCM** or **AES-CCM** MAY be used where hardware AES exists. Implementations **MUST** use a vetted library (e.g. Monocypher) and **MUST NOT** hand-roll the primitive.

11.4 Nonce construction

The 96-bit nonce is derived deterministically — never random, never fully transmitted:

nonce[12] = dir(1) sysid(1) compid(1) seq(1) timestamp(6) 0x00 0x00

dir = 0 for FC→GCS, 1 for GCS→FC (the two directions never share a nonce under one key); sysid, compid, seq come from the cleartext header; timestamp is the trailer field. Uniqueness holds while (timestamp, seq) never repeats for a (dir, sysid, compid) — guaranteed because timestamp is monotonic and a 100 μ s tick is finer than 256 seq values at any real rate. A sender **MUST NOT** reuse a nonce under a key.

11.5 Replay protection (reboot-safe)

A receiver **MUST** accept a secured frame only if its timestamp is **both**:

1. within the window [now - skew, now] against the receiver's synced clock, and
2. strictly greater than the last timestamp accepted from that (sysid, compid, direction).

skew is a configured tolerance (default 1 s for a UART link). Because the anchor is the *synced* clock (not a free counter), a power-cycle cannot rewind it: on reconnect the FC re-syncs (§10.5) and any frame captured from a past session is outside the window and rejected — **no counter is persisted to flash**.

11.6 Order of operations

Sender: truncate trailing-zero **plaintext** (§5.6) → set **payload_len** → encrypt the truncated plaintext (stream cipher preserves length) → compute tag over (header ciphertext) with the §11.4 nonce → emit. Receiver: verify tag → decrypt → zero-fill to full message length → decode. **payload_len** is in the authenticated header, so the fill length cannot be tampered.

11.7 Keys

A **pre-shared key** lives in 0:navlink.key on the FC SD card, read once at boot, never transmitted; the GCS holds the matching key in config. Provisioning is out of band (write the card directly). **CAPABILITIES.key_id** is a **non-secret** identifier so both ends confirm a shared key at connect rather than failing silently. Key rotation is permitted only over USB or an already-secured link. An **X25519 (ECDH) handshake** for per-session keys/forward secrecy MAY be added later, negotiated via **CAPABILITIES**; not required for the first cut.

11.8 Downgrade protection

A peer that sets a security flag the receiver did not negotiate is dropped (the flags are `incompat`, §3.3). A vehicle MAY be configured to **reject** an unsecured `CMD_ARM` (and any other safety-critical command), so a forced downgrade to cleartext cannot arm it.

12. Services

All services are ordinary messages (one ID space). Telemetry is best-effort; these are the request/response/transactional patterns.

12.1 Command service

A command is a **typed message** in the command range (§9), opening with the shared header { `target_sys:u8`, `target_comp:u8`, `req_seq:u8` } followed by typed parameters (no float box, no `COMMAND_LONG`/`COMMAND_INT` split). The receiver answers with `COMMAND_ACK` { `command:u24`, `req_seq:u8`, `result`, `progress`, `result_param2` }, correlated by (command msgid, req_seq). An FC that is unsynchronised (§10.5) MUST reject commands.

12.2 Parameter service

- `PARAM_VALUE` { `param_id:char[16]`, `index:u16`, `count:u16`, `generation:u32`, `type:u8`, `value:u8[8]` } — `value` is the real typed value (8 B, LE), never a float bit-cast. Metadata (type/unit/scale/range/default) comes from the dialect.
- **Transactional dump:** every value carries **generation** (bumped on any `PARAM_SET`). A complete set is all count indices under one **generation**; a mid-dump generation change forces re-sync; missing indices are re-requested by `PARAM_REQUEST_READ` { `index` }.
- **Confirmed writes:** a `PARAM_SET` is answered by the updated `PARAM_VALUE` carrying the new **generation**.
- Also: `PARAM_REQUEST_LIST`, `PARAM_REQUEST_READ` { `param_id` | `index` }, `PARAM_SET` { `param_id`, `type`, `value` }.

12.3 Mission service

`MISSION_COUNT`, `MISSION_REQUEST_INT` { `seq` }, `MISSION_ITEM_INT` { `seq`, `frame`, `command`, `x`, `y`, `z`, ... }, `MISSION_ACK`, `MISSION_CURRENT`, `MISSION_CLEAR_ALL`. A mission carries a **generation** and **commits atomically**: the FC applies it only once all `MISSION_COUNT` items have arrived and validated; a partial upload never commits. `MISSION_ACK` returns the committed **generation**.

12.4 Bulk transfer

`FILE_TRANSFER` { `session:u8`, `offset:u32`, `len:u8`, `data:u8[]` } with windowed ACKs, for SD-card log download (0:v_nav.bin) and blob upload. Large chunks MAY set `IFLAG_CRC32` (§3.3) so the stronger 4-byte CRC-32 protects multi-kB payloads.

12.4.1 Fragmentation (IFLAG_FRAGMENTED) — **reserved** Frame-level fragmentation of payloads larger than 255 bytes is reserved behind `IFLAG_FRAGMENTED`; its header extension and reassembly rules are **not yet specified** in this version. Until specified, bulk data uses the application-level `FILE_TRANSFER` windowing above.

13. Capability negotiation

In place of a per-frame `compat_flags` byte, capabilities are exchanged once at connect:

- `HEARTBEAT.capabilities` — a u32 bitmask of coarse capabilities, sent ~1 Hz.
- `CAPABILITIES { protocol_version:u8, incompat_supported:u32, msgid_ranges:..., sec_modes:u8, key_id:u32 }` — the full list, on request at connect. `sec_modes` advertises `none/sign/encrypt`; `key_id` is the non-secret key identifier (§11.7); `incompat_supported` lets a peer learn which `incompat` bits the other implements before relying on them.

A peer **MUST NOT** rely on an `incompat` feature the other has not advertised.

14. Message catalog

IDs are normative block bases; codegen assigns exact values within a block. Field-level detail is in the dialect (§7); this is the index.

14.1 Core / system (0–255)

msgid	message	key fields
0	HEARTBEAT	type, autopilot, base_mode, system_status, nav_state, capabilities, timestamp [s:20\ms:12]
1	SYS_STATUS	sensors_present/enabled/health
2	SYSTEM_HEALTH	bitmasks, load, voltage
3	FLIGHT_MODE	tx_overflow, imu_drop, log_wrap, cpu_load
4	STATUSTEXT	mode, source
5	COMMAND_ACK	severity, text[50], id, chunk_seq
8	PING	command:u24, req_seq, result, progress, result_param2
9	CAPABILITIES	seq, target
10	TIME_REFERENCE	protocol_version, incompat_supported, msgid_ranges, sec_modes, key_id
11	TIME_REFERENCE_ACK	epoch_unix_s, gcs_send_us
		gcs_send_us, fc_recv_us, fc_send_us

14.2 Sensors / state (1024–2047)

msgid	message	key fields
1024	IMU_RAW	acc[3], gyr[3], mag[3], temp, sample_time_us
1025	IMU_COMPRESSED	f16 deltas + ref seq
1026	ATTITUDE_EULER	roll, pitch, yaw, rollspeed, pitchspeed, yawspeed
1027	ATTITUDE_QUATERNION	q[4], rollspeed, pitchspeed, yawspeed (NED)
1028	RC_CHANNELS	chan[18]:u16, rssi, count
1029	MOTOR_TELEMETRY	cmd[8]
1030	CONTROL_TRACE	18×f32 PID/loop trace
1031	VIBRATION	vib_x/y/z, clip counts
1032	SCALED_PRESSURE	abs/diff pressure, temp

14.3 Navigation / position (2048–3071)

msgid	message	key fields (fixed-point per §5.4)
2048	GPS_RAW_INT	fix_type, lat(degE7), lon(degE7), alt(mm), eph, epv, vel, cog, satellites
2049	GLOBAL_POSITION_INT	lat, lon, alt, rel_alt, vx, vy, vz, hdg
2050	LOCAL_POSITION_NED	x, y, z, vx, vy, vz
2051	HOME_POSITION	lat, lon, alt
2052	ALTITUDE	alt_monotonic, alt_amsl, alt_local, alt_terrain
2053	ESTIMATOR_STATUS	flags, pos/vel/hgt ratios

14.4 Power / propulsion (3072–4095)

msgid	message	key fields
3072	BATTERY_STATUS	id, voltages[10], current, consumed_mAh, remaining%, temp
3073	ESC_TELEMETRY	per-esc rpm, voltage, current, temp
3074	POWER_STATUS	Vcc, Vservo, flags

14.5 Peripherals / payload (4096–8191)

GIMBAL_DEVICE_ATTITUDE_STATUS, CAMERA_TRIGGER, OBSTACLE_DISTANCE, COLLISION, ADSB_VEHICLE, TUNNEL (vendor passthrough), DEBUG_VECT / NAMED_VALUE_FLOAT (schema-free dev telemetry).

14.6 Commands (8192–12287)

Each opens with { target_sys, target_comp, req_seq }; acked by COMMAND_ACK.

msgid	command	params
8192	CMD_ARM	force:u8
8193	CMD_DISARM	force:u8
8194	CMD_CALIBRATE_IMU	which:u8
8195	CMD_SET_PID	axis:u8, kp:f32, ki:f32, kd:f32
8196	CMD_SET_GYRO_LPF	cutoff_hz:u16
8197	CMD_SET_MOTOR_GEOMETRY	layout:u8, n:u8, mix[:f32
8198	CMD_SET_FLIGHT_MODE	mode:u8, source:u8

14.7 Services (12288–16383)

msgid	block	service
12288–12299	PARAM_*	§12.2
12300–12311	MISSION_*	§12.3
12312–12319	FILE_TRANSFER	§12.4
12320–12329	calibration progress/instructions	—

15. Migration (v1 → v2)

A dual parser keys on the version byte (§3.4); v1 (`byte1 & 0x0F == 1`) and v2 (`byte1 == 0x02`) coexist on one UART with no flag day.

Phase	Content
0	Author <code>dialect.json</code> (incl. all v1 messages); ship <code>generate.py</code> ; generated v1 codec byte-identical to legacy (golden vectors). Kills the three hand codecs.
1	v2 framing behind <code>ENABLE_NAVLINK_V2</code> ; dual parser; add <code>seq/sysid/compid/CRC-16+CRC_EXTRA/incompat</code> byte; capability exchange; link stats from <code>seq</code> gaps.
2	Dual-emit telemetry (v1+v2), GCS prefers v2; move the seven <code>SYSTEM_STATUS</code> sub-messages to first-class v2 msgids; validate parity on SITL + flight.
3	Typed command/param/mission services + <code>COMMAND_ACK</code> ; shim legacy <code>cmd_id</code> →command msgid; autotuner waits on <code>COMMAND_ACK/PARAM_VALUE</code> .
4	Drop v1 emit + v1 parse (keep v1 decoder for old logs); bump <code>incompat</code> baseline.

Touch-points. FC: `comm_types.h`, `serializer.c`, `deserializer.c`, `comm_processor.c`, `telemetry_task.c`, `variables.h`. GCS: `DroneProtocol.{cpp,h}`, `PacketDecoder.{cpp,h}`, `core/Types.h`, `crc.cpp`. Tools: `tools/autotune/protocol.py`. New: `navlink/dialect.json` (+ schema), `tools/navlink/generate.py`, generated codecs in all three trees.

16. Conformance & test vectors

An implementation is conformant if it (a) frames/parses per §3, (b) computes CRC/CRC_EXTRA per §4, (c) encodes fields per §5–§6, (d) generates from the dialect per §7–§8, (e) implements the time gate §10.5 and security §11 when crypto is enabled, and (f) passes the vectors below.

16.1 CRC-16/MCRF4XX self-check

`crc16("123456789")` (the nine ASCII digits) MUST equal `0x6F91`.

16.2 CRC_EXTRA worked example

For `ATTITUDE_EULER` (§7.3) the `CRC_EXTRA` input string is:

```
"ATTITUDE_EULER " "f32 roll " "f32 pitch " "f32 yaw "
"f32 rollspeed " "f32 pitchspeed " "f32 yawspeed "
```

(no array-length bytes — no arrays; extension fields excluded). Implementations MUST agree on the resulting `CRC_EXTRA` byte; codegen emits it into the table and the golden-vector test pins it.

16.3 Unsecured frame round-trip

A `HEARTBEAT` (msgid 0, 7-field payload of §7.3) from `sysid=1`, `compid=1`, `seq=0` MUST:

1. serialise to `0x56 0x02 <len> 0x00 0x00 0x01 0x01 0x00 0x00 0x00 <payload> <crc_lo> <crc_hi>`;
2. round-trip through `pack/unpack/to_aligned` to an identical aligned struct;

3. survive trailing-zero truncation (drop trailing-zero `nav_state=0...` bytes, shorten `payload_len`, zero-fill on receive) with an identical decode.

16.4 Secured frame

With `IFLAG_ENCRYPTED`, a frame MUST: carry no CRC; carry `[timestamp:6] [tag:16]`; use the §11.4 nonce; verify the tag over (`header[1..9]` ciphertext); be rejected if `timestamp` is outside `[now-skew, now]` or not strictly increasing; be rejected entirely if the FC is unsynchronised (§10.5).

16.5 Version demux

Bytes `0x56 0x31 ...` (v1: type 3, version 1) MUST route to the v1 parser; `0x56 0x02 ...` to v2; `0x56 0x07 ...` MUST trigger resync.

17. Appendix — reference enumerations

```
/* incompat_flags */
enum { IFLAG_SIGNED = 0x01, IFLAG_ENCRYPTED = 0x02,
        IFLAG_CRC32 = 0x04, IFLAG_FRAGMENTED = 0x08 };

/* COMMAND_ACK.result */
enum { RESULT_ACCEPTED = 0, RESULT_TEMPORARILY_REJECTED = 1, RESULT_DENIED = 2,
        RESULT_UNSUPPORTED = 3, RESULT_FAILED = 4, RESULT_IN_PROGRESS = 5 };

/* PARAM_VALUE.type */
enum { PT_U8=1, PT_I8=2, PT_U16=3, PT_I16=4, PT_U32=5, PT_I32=6,
        PT_U64=7, PT_I64=8, PT_F32=9, PT_F64=10 };

/* security modes (CAPABILITIES.sec_modes bitmask) */
enum { SEC_NONE = 0x01, SEC_SIGN = 0x02, SEC_ENCRYPT = 0x04 };
```

End of specification.